Week 12 and 13 DML for Conditional Average Treatment Effect

We demonstrate the use of Double Machine Learning to estimate the Conditional Average Treatment Effect. The goal of the example is to estimate the effect of 401(k) eligibility on net financial assets for different values of income both parametrically and non-parametrically.

Parametric Estimation Background

The target function is Conditional Average Treatment Effect, defined as

$$g(x) = E[Y(1) - Y(0) \mid X = x],$$

where $Y(1)$ and $Y(0)$ are potential outcomes in treated and control group. In our case, $Y(1)$ is the potential Net Total Financial Assets (Net TFA) if a subject is eligible for 401(k), and $Y(0)$ is the potential Net TFA if a subject is ineligible. X is a covariate of interest, in this case, income. $g(x)$ shows expected effect of eligibility on NFA for a subject whose income level is x.

If eligibility indicator is independent of $Y(1), Y(0)$, given pre-401(k) assignment characteristics Z, the function can expressed in terms of observed data. Observed data consists of realized NFA $Y = DY(1) + (1-D)Y(0)$, eligibility indicator D, and covariates Z which includes X, income. The expression for $g(x)$ is

$$g(x) = E[Y(\eta_0) \mid X = x]$$

where the transformed outcome variable is

$$Y(\eta) = \frac{D}{s(Z)}\big(Y - \mu(1,Z)\big) - \frac{1-D}{1-s(Z)}\big(Y - \mu(0,Z)\big) + \mu(1,Z) - \mu(0,Z)$$

the probability of eligibility is

$$s_0(z) = Pr(D = 1 \mid Z = z)$$

the expected net financial asset given $D = d \in \{1, 0\}$ and $Z = z$ is

$$\mu_0(d, z) = E[Y \mid Z = z, D = d]$$

Our goal is to estimate $g(x)$.

In step 1, we estimate the unknown functions $s_0(z)$, $\mu_0(1,z)$, $\mu_0(0,z)$ and plug them into $Y(\eta)$. In step 2, we approximate the function $g(x)$ by a linear combination of basis functions:

$$g(x) = p(x)'\beta_0$$

where $p(x)$ is a vector of group indicators, polynomials, or splines and

$$\beta_0 = (E[p(X)p(X)'])^{-1}E[p(X)Y(\eta_0)]$$

is the best linear predictor. We report

$$\hat{g}(x) = p(x)'\hat{\beta},$$

where $\hat{\beta}$ is the ordinary least squares estimate of β_0 defined on the random sample $(X_i, D_i, Y_i)_{i=1}^N$

$$\hat{\beta} := \left(\frac{1}{N}\sum_{i=1}^{N} p(X_i)p(X_i)'\right)^{-1} \frac{1}{N}\sum_{i=1}^{N} p(X_i)Y_i(\hat{\eta})$$

DML for Doubly Robust Conditional ATE

```
library(foreign)
library(quantreg)
library(splines)
library(lattice)
library(Hmisc)
library(fda)
library(grf)
library(hdm)
library(randomForest)
library(ranger)
library(sandwich)
library(dplyr)
library(ggplot2)
```

```r
## 401k dataset
data(pension)
pension$net_tfa <- pension$net_tfa / 10000

## covariate of interest is pre-treatment log income
pension$inc <- log(pension$inc)
pension <- pension[!is.na(pension$inc) & pension$inc != -Inf & pension$inc != Inf, ]


## outcome variable is net total financial assets
Y <- pension$net_tfa

## binary treatment is indicator of 401(k) eligibility
D <- pension$e401

## the variable where ATE is conditioned on is pre-treatment log income
X <- pension$inc

## raw covariates so that Y(1) and Y(0) are independent of D given Z
Z <- pension[, c(
  "inc", "age", "fsize", "educ", "male", "db", "marr", "twoearn", "pira", "hown", "hval
  "nohs", "hs", "smcol"
)]
cat(sprintf("\n sample size is %g \n", length(Y)))
```

```
 sample size is 9910
```

```r
cat(sprintf("\n num raw covariates z is %g \n", dim(Z)[2]))
```

```
 num raw covariates z is 16
```

In Step 1, we estimate three functions:

1. probability of treatment assignment $s_0(z)$; 2 and 3. regression functions $\mu_0(1, z)$ and $\mu_0(0, z)$. We use the cross-fitting procedure with $K = 2$ holds. For each function, we try random forest as a demonstration.

```
## define the first stage function that estimates the nuisance function
first_stage_rf <- function(Y, D, Z, seed = 1) {

  ## sample size
  N <- length(D)

  ## estimated regression function in control group
  mu0_hat <- rep(1, N)

  ## estimated regression function in treated group
  mu1_hat <- rep(1, N)

  ## propensity score
  s_hat <- rep(1, N)

  ## define sample splitting
  set.seed(seed)
  inds_train <- sample(1:N, floor(N / 2))
  inds_eval <- setdiff(1:N, inds_train)

  ## conditional probability of 401 k eligibility (i.e., propensity score) based on r
  Df <- as.factor(as.character(D))
  fitted_rf_pscore <- randomForest(Z, Df, subset = inds_train)
  s_hat[inds_eval] <- predict(fitted_rf_pscore, Z[inds_eval, ], type = "prob")[, 2]
  fitted_rf <- randomForest(Z, Df, subset = inds_eval)
  s_hat[inds_train] <- predict(fitted_rf_pscore, Z[inds_train, ], type = "prob")[, 2]

  ## conditional expected net financial assets (i.e., regression function) based on r
  covariates1 <- cbind(Z, D = rep(1, N))
  covariates0 <- cbind(Z, D = rep(0, N))

  fitted_rf_mu <- randomForest(cbind(Z, D), Y, subset = inds_train)
  mu1_hat[inds_eval] <- predict(fitted_rf_mu, covariates1[inds_eval, ])
```

```
  mu0_hat[inds_eval] <- predict(fitted_rf_mu, covariates0[inds_eval, ])

  fitted_rf_mu <- randomForest(cbind(Z, D), Y, subset = inds_eval)
  mu1_hat[inds_train] <- predict(fitted_rf_mu, covariates1[inds_train, ])
  mu0_hat[inds_train] <- predict(fitted_rf_mu, covariates0[inds_train, ])

  return(list(
    mu1_hat = mu1_hat,
    mu0_hat = mu0_hat,
    s_hat = s_hat
  ))
}
```

Parametric Estimation

In Step 2, we approximate $Y(\eta_0)$ by a vector of basis functions. The simplest use case of Conditional Average Treatment Effect is GATE, or Group Average Treatment Effect. Partition the support of income as

$$-\infty = \ell_0 < \ell_1 < \ell_2 \ldots \ell_K = \infty$$

define intervals $I_k = [\ell_{k-1}, \ell_k)$. Let X be income covariate. For X, define a group indicator

$$G_k(X) = [X \in I_k]$$

and the vector of basis functions

$$p(X) = (G_1(X), G_2(X), \ldots, G_K(X))$$

Then, the Best Linear Predictor β_0 vector shows the average treatment effect for each group.

```
## estimate first stage functions by random forest
## may take a while
fs_hat_rf <- first_stage_rf(Y, D, Z)

X <- pension$inc
```

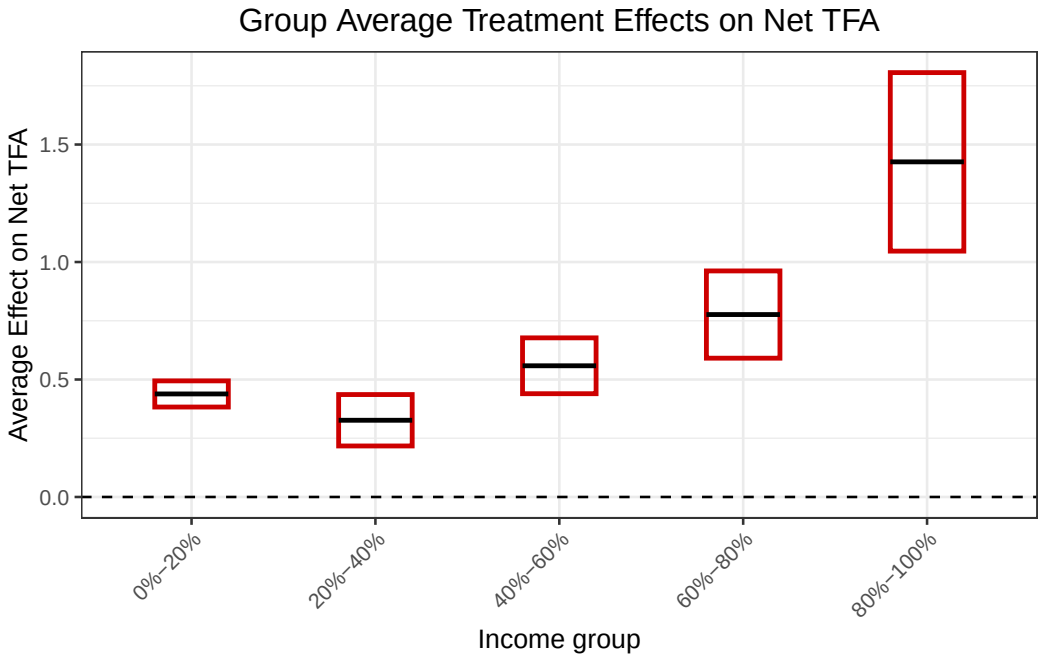
```r
fs_hat <- fs_hat_rf

## regression function
mu1_hat <- fs_hat[["mu1_hat"]]
mu0_hat <- fs_hat[["mu0_hat"]]

## propensity score
s_hat <- fs_hat[["s_hat"]]
min_cutoff <- 0.01
s_hat <- sapply(s_hat, max, min_cutoff)

## construct orthogonal or robust signal
RobustSignal <- (Y - mu1_hat) * D / s_hat - (Y - mu0_hat) * (1 - D) / (1 - s_hat) + mu1
```

```r
## define the function of group-specific ATE
group_average_treatment_effect <- function(X, Y, max_grid = 5, alpha = 0.05) {

  ## calculate income quintile
  grid <- quantile(X, probs = c((0:max_grid) / max_grid))
  Xraw <- matrix(NA, nrow = length(Y), ncol = length(grid) - 1)

  ## create dummies for income quintile
  for (k in 2:length(grid)) {
    Xraw[, k - 1] <- sapply(X, function(x) ifelse(x >= grid[k - 1] & x < grid[k], 1, 0)
  }

  ## fit regression
  ols_fit <- lm(Y ~ Xraw - 1)
  coefs <- coef(ols_fit)
  vars <- names(coefs)
  hcv_coefs <- vcovHC(ols_fit, type = "HC")
  coefs_se <- sqrt(diag(hcv_coefs)) ## White's standard sandwich estimator

  ## confidence interval
  tes_uci <- coefs + qnorm(1 - alpha / 2) * coefs_se
  tes_lci <- coefs + qnorm(alpha / 2) * coefs_se
```

```r
  return(list(
    beta_hat = coefs, ghat_lower_point = tes_lci, ghat_upper_point = tes_uci
  ))
}

## predict beta and confidence interval
res <- group_average_treatment_effect(X = X, Y = RobustSignal)
```

```r
## plot CATE by income
data.frame(
  group = factor(c("0%-20%", "20%-40%", "40%-60%", "60%-80%", "80%-100%"),
                 levels = c("0%-20%", "20%-40%", "40%-60%", "60%-80%", "80%-100%")),
  tes = res$beta_hat,
  lci = res$ghat_lower_point,
  uci = res$ghat_upper_point
) %>%
  ggplot(aes(x = group, y = tes)) +

  ## CI rectangles (95% intervals)
  geom_rect(aes(xmin = as.numeric(group) - 0.2,
                xmax = as.numeric(group) + 0.2,
                ymin = lci, ymax = uci),
            fill = NA, color = "red3", linewidth = 0.85) +

  ## point estimate
  geom_segment(aes(x = as.numeric(group) - 0.2,
                   xend = as.numeric(group) + 0.2,
                   y = tes, yend = tes),
               linewidth = 0.85) +
  # zero line
  geom_hline(yintercept = 0, color = "black", linetype = "dashed") +
  labs(
    title = "Group Average Treatment Effects on Net TFA",
    x = "Income group",
```

```
    y = "Average Effect on Net TFA"
  ) +
  theme_bw(base_size = 10) +
  theme(
    plot.title = element_text(hjust = 0.5),
    axis.text.x = element_text(angle = 45, hjust = 1)
  ) +
  scale_x_discrete(drop = FALSE)
```



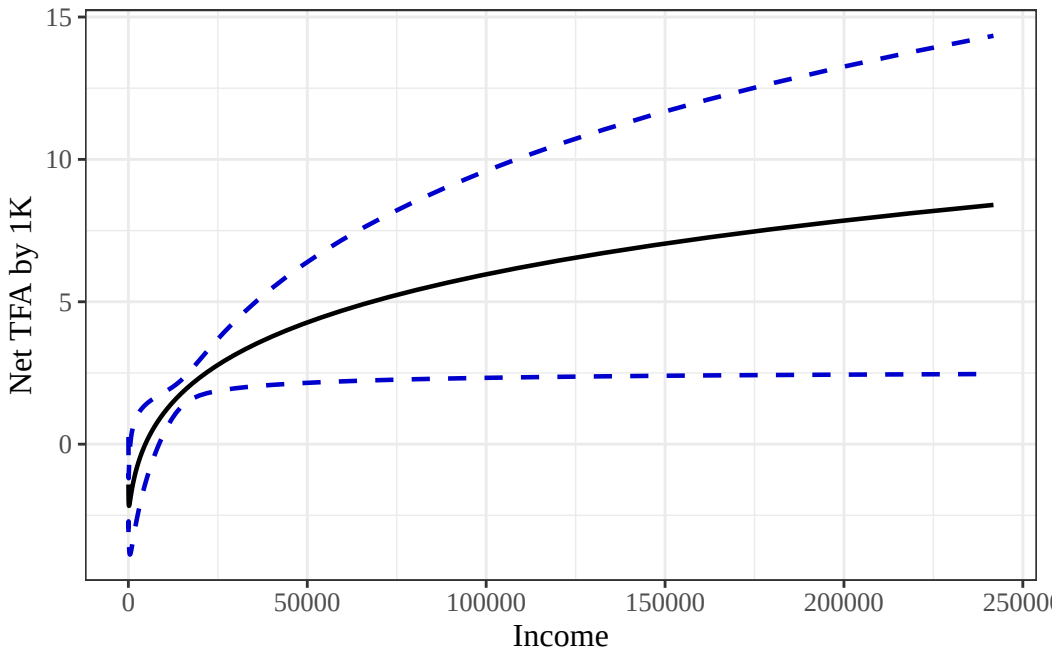
Fit CATE by Income Polynomials

```
## least squares series
least_squares_series <- function(X, Y, max_degree=3) {
  cv_pol <- rep(0, max_degree)
  for (degree in 1:max_degree) {
    formula.pol <- Y ~ poly(X, degree)
    fit <- lm(formula.pol)
    cv_pol[degree] <- sum((fit$res / (1 - hatvalues(fit)))^2)
  }
  ## number of polynomials chosen by cross-validation
```

```r
  cv_degree <- which.min(cv_pol)

  ## estimate coefficients
  formula.pol <- Y ~ poly(X, cv_degree)
  fit <- lm(formula.pol)

  return(list(fit = fit, cv_degree = cv_degree))
}

## second stage
second_stage <- function(X, Y = RobustSignal, norder = 4, nderiv = 0,
                         alpha = 0.05, eps = 0.1) {
  x_grid <- seq(min(X), max(X), eps)
  res <- least_squares_series(X = X, Y = RobustSignal, max_degree = 3)
  fit <- res$fit
  cv_degree <- res$cv_degree
  regressors_grid <- cbind(rep(1, length(x_grid)), poly(x_grid, cv_degree))
  g_hat <- regressors_grid %*% coef(fit)
  hcv_coefs <- vcovHC(fit, type = "HC")
  standard_error <- sqrt(diag(regressors_grid %*% hcv_coefs %*% t(regressors_grid)))

  ## lower pointwise CI
  ghat_lower_point <- g_hat + qnorm(alpha / 2) * standard_error
  ## upper pointwise CI
  ghat_upper_point <- g_hat + qnorm(1 - alpha / 2) * standard_error

  return(list(
    g_hat = g_hat, fit = fit,
    ghat_lower_point = ghat_lower_point, ghat_upper_point = ghat_upper_point, x_grid =
  ))
}
```

```r
## fit second stage
res_rf_least_squares_series <- second_stage(X = X, Y = RobustSignal)
data.frame(income = rep(res_rf_least_squares_series$x_grid, 3),
                    outcome = c(res_rf_least_squares_series$ghat_lower_point,
```

```
                                  res_rf_least_squares_series$g_hat,
                                  res_rf_least_squares_series$ghat_upper_point),
          group = c(rep("lower", length(res_rf_least_squares_series$x_grid)),
                    rep("estimate", length(res_rf_least_squares_series$x_grid)),
                    rep("upper", length(res_rf_least_squares_series$x_grid)))) %>%
ggplot() +
    aes(x = exp(income), y = outcome, group = group) +
    geom_line(aes(linetype = group, color = group), size = 0.8) +
    theme_bw() +
    xlab("Income") +
    ylab("Net TFA by 1K") +
    scale_colour_manual(values = c("black", "blue3", "blue3")) +
    theme(plot.title = element_text(hjust = 0.5),
          text = element_text(size = 12, family = "serif")) +
    theme(legend.title = element_blank()) +
    theme(legend.position = "none") +
    scale_linetype_manual(values = c("solid","dashed","dashed"))
```



Non-Parametric Estimation

Here we demonstrate the use of non-parametric estimation of HTE, specifically Doubly Robust Forest and Causal Forest (see similar figures on pp. 382).

```r
## here we regress robust signal against Z that includes X
dr.forest <- regression_forest(Z, RobustSignal)

## create median-fixed grid for prediction
medians <- as.list(apply(Z[, setdiff(names(Z), "inc")], 2, median))
centers <- kmeans(Z$inc, centers = 10)$centers
inc.grid <- sort(as.numeric(centers))
## I use kmean clustering to minimize within-cluster variance
## it represenst 10 typical income levels, spread so that dense regions of inc get mo

## data for prediction
newdata <- data.frame(
  inc = inc.grid,
  matrix(rep(unlist(medians), each = length(inc.grid)),
         nrow = length(inc.grid), byrow = FALSE)
)
names(newdata) <- names(Z)

## predict from DR forest with variance estimates for CI
pred <- predict(dr.forest, newdata = newdata, estimate.variance = TRUE)

## build tidy data frame for plotting
plot_df <- data.frame(
  exp_inc  = exp(newdata$inc),
  tau_hat  = pred$predictions,
  ci_lower = pred$predictions - 1.96 * sqrt(pred$variance.estimates),
  ci_upper = pred$predictions + 1.96 * sqrt(pred$variance.estimates)
)

# plot
DR_forest <- ggplot(plot_df, aes(x = exp_inc, y = tau_hat)) +
  geom_ribbon(aes(ymin = ci_lower, ymax = ci_upper),
```

```
              fill = "grey", alpha = 0.3) +
  geom_line(color = "red4", linewidth = 1) +
  labs(
    x = "Income",
    y = "Estimated CATE (conditional at medians of other features)",
    title = "Conditional CATE by income (DR Forest)"
  ) +
  theme_bw(base_size = 10) +
  theme(
    plot.title = element_text(hjust = 0.5)
  )
```

Below we fit the same CATE estimates using Causal Forest.

```
## causal forest
cf <- causal_forest(Z, Y, D,
                    num.trees = 2000,
                    honesty = TRUE)

## build grid for prediction (same as before)
medians <- as.list(apply(Z[, setdiff(names(Z), "inc")], 2, median))
centers <- kmeans(Z$inc, centers = 10)$centers
inc.grid <- sort(as.numeric(centers))
newdata <- data.frame(
  inc = inc.grid,
  matrix(rep(unlist(medians), each = length(inc.grid)),
         nrow = length(inc.grid), byrow = FALSE)
)
names(newdata) <- names(Z)

## predict CATEs with variance estimates
pred <- predict(cf, newdata = newdata, estimate.variance = TRUE)

## build tidy data frame for plotting
plot_df <- data.frame(
  exp_inc  = exp(newdata$inc),
```

```
  tau_hat  = pred$predictions,
  ci_lower = pred$predictions - 1.96 * sqrt(pred$variance.estimates),
  ci_upper = pred$predictions + 1.96 * sqrt(pred$variance.estimates)
)

## plot
causal_forest <- ggplot(plot_df, aes(x = exp_inc, y = tau_hat)) +
  geom_ribbon(aes(ymin = ci_lower, ymax = ci_upper),
              fill = "grey", alpha = 0.3) +
  geom_line(color = "blue4", linewidth = 1) +
  labs(
    x = "Income",
    y = "Estimated CATE (conditional at medians of other features)",
    title = "Conditional CATE by income (Causal Forest)"
  ) +
  theme_bw(base_size = 10) +
  theme(plot.title = element_text(hjust = 0.5))

library(ggpubr)
ggarrange(DR_forest, causal_forest)
```